Mini Top Utility with Visualization

A Complete C++ System Monitoring Tool

Comprehensive Code Walkthrough & Presentation

Pidatala Anvesh Kiran CS2517

Saikat Jana CS2524

Indian Statistical Institute (ISI)

M.Tech in Computer Science

Language:	`C++17`
UI Library:	ncurses (Terminal-Based Interface)
Data Source:	Linux `/proc` Virtual Filesystem
Platform:	Linux (Ubuntu / CentOS / Fedora)
Build Tool:	`g++ -std=c++17 -lncurses`

April 15, 2026

Contents

1 Project Overview **3**
1.1 Objective . . . 3
1.2 What Is This Project? . . . 3
1.3 How Does It Work? . . . 3

2 Code Part 1: Header Files and Namespace **5**
2.1 What Each Header Provides . . . 5
2.2 The Namespace Alias . . . 6

3 Code Part 2: Data Structures (Structs) **6**
3.1 Structure 1: SystemSummary . . . 6
3.2 Structure 2: TaskStatus . . . 7
3.3 Structure 3: MemoryStats . . . 8
3.4 Structure 4: CPUData (New) . . . 8
3.5 Structure 5: Process . . . 9

4 Code Part 3: Step 1 — System Summary Functions **9**
4.1 Function: getupTime() — Reading System Uptime . . . 9
4.2 Function: getLoadAverage() — Reading CPU Load Averages . . . 9
4.3 Function: getActiveUserCount() — Counting Logged-In Users . . . 10
4.4 Function: getDetailedTaskCount() — Counting Process States . . . 10
4.5 Function: getMemoryAndSwap() — Reading Memory Statistics . . . 11

5 Code Part 4: Step 2 — Live CPU Usage Measurement **12**
5.1 Why CPU Usage Requires Two Readings . . . 12
5.2 The CPUData Structure . . . 12
5.2.1 Field-by-Field Explanation . . . 13
5.2.2 The Two Helper Methods . . . 13
5.3 Function: readCPUStats() — Reading /proc/stat . . . 13
5.4 Numerical Example of the Delta Calculation . . . 13

6 Code Part 5: Step 3 — Per-Process Information **14**
6.1 The Process Structure . . . 14
6.2 Function: getProcessTicks() — Measuring Individual Process CPU Time 15
6.3 Function: getUsername() — Converting User IDs to Names . . . 15
6.4 Function: getProcessList() — The Main Process Data Collector . . . 15

7 Code Part 6: The main() Function — Where Everything Comes Together **17**
7.1 Part 1: Initializing ncurses (Terminal UI Setup) . . . 17
7.2 Part 2: One-Time Setup — Getting Total System Memory . . . 17
7.3 Part 3: Declaring Main Loop Variables . . . 17
7.3.1 Variable-by-Variable Explanation . . . 17
7.4 Part 4: The Main Loop — Section A: User Input Handling . . . 18
7.4.1 Explanation . . . 18
7.4.2 The ‘d’ Key: Change Refresh Delay . . . 18

7.4.3 The ‘k’ Key: Kill Process by PID 19
7.4.4 The ‘n’ Key: Change Number of Displayed Processes 20
7.4.5 The ‘m’ Key: Toggle Sort by Memory Usage 21
7.5 Part 5: The Main Loop — Section B: Data Collection 21
7.5.1 Step 1 Data: System Summary 21
7.5.2 Step 2 Data: Overall CPU Percentage 21
7.5.3 Explanation with Formulas 22
7.5.4 Step 3 Data: Per-Process Information 22
7.6 Part 6: The Main Loop — Section C: Screen Rendering 23
7.7 Part 7: The Main Loop — Section D: Preparing for Next Cycle 24
7.8 Part 8: Program Cleanup 24

8 Compilation and Execution **25**
8.1 Build Command 25
8.2 Run Command 25
8.3 Runtime Controls 25

1 Project Overview

1.1 Objective

To build a real-time, interactive terminal dashboard that monitors Linux system performance and process resource allocation and to visualize system performance using terminal UI.

1.2 What Is This Project?

This project is a custom implementation of the Linux `top` command. It is a real-time system monitoring tool. It provides a dynamic, terminal-based dashboard that tracks the health and performance of a Linux operating system. A Linux system monitor that replicates and enhances the traditional `top` command with real-time visual display of:

- System summary (Current time, System uptime, Number of users, Load average)
- Task/process statistics (Running / Sleeping / Zombie)
- Memory and swap usage (total, free, used, cache)
- CPU usage with breakdown (us, sy, id)
- Live process list sorted by CPU usage (PID USER PR NI VIRT RES %CPU %MEM COMMAND)

. The `top` command is one of the most fundamental system administration tools on Linux — it provides a **real-time, auto-refreshing dashboard** that shows:

- How long the system has been running (uptime)
- How many users are currently logged in
- How busy the CPU is (broken down by category)
- How much RAM and swap memory is in use
- A sorted list of all running processes with their resource consumption

1.3 How Does It Work?

Linux exposes all system and process information through a special virtual filesystem called `/proc`. Our program:

1. **Reads** raw data from files in `/proc`
2. **Parses** the text data into structured information
3. **Calculates** derived metrics (CPU%, MEM%, etc.)
4. **Displays** everything in a live terminal dashboard using `ncurses`
5. **Repeats** every second (user-configurable)

Keywords

Abbreviation	Full Name	Meaning
us	User	Time spent running normal user programs (like Firefox, Python scripts.
sy	System	Time spent running kernel/OS code (like managing files, network)
ni	Nice	Time spent running low-priority user programs
id	Idle	Time the CPU was doing absolutely nothing (99.7% means CPU is almost free)
wa	Wait	Time CPU was idle because it was waiting for disk I/O
hi	Hardware Interrupt	Time handling hardware signals
si	Software Interrupt	Time handling software signals
st	Steal	Time "stolen" by a virtual machine host (relevant in cloud/VMs)

The /proc Filesystem

`/proc` is **not a real filesystem on disk**. It is a **virtual filesystem** created by the Linux kernel in memory. Every "file" in `/proc` is actually the kernel exposing live system data in a text format that programs can easily read.

- `/proc/uptime` — System uptime in seconds
- `/proc/loadavg` — CPU load averages
- `/proc/stat` — CPU time counters
- `/proc/meminfo` — Memory statistics
- `/proc/[PID]/` — One directory per running process

2 Code Part 1: Header Files and Namespace

Every C++ program begins by including the libraries it needs. Each `#include` brings in specific functionality.

```cpp
#include <iostream>
#include <fstream>
#include <string>
#include <sstream>
#include <vector>
#include <filesystem>
#include <iomanip>
#include <utmp.h>
#include <thread>
#include <chrono>
#include <ncurses.h>
#include <dirent.h>
#include <pwd.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>
#include <algorithm>
#include <map>

namespace fs = std::filesystem;
```

Listing 1: Header file inclusions

2.1 What Each Header Provides

Header	Purpose
`<iostream>`	Standard I/O streams (`cout`, `cin`). Basic C++ output capability.
`<fstream>`	File stream classes (`ifstream`, `ofstream`). Used to **open and read** files like `/proc/uptime`, `/proc/meminfo`, etc.
`<string>`	The `std::string` class. Used everywhere for text handling — storing usernames, commands, file contents.
`<sstream>`	`std::stringstream` class. Lets us treat a string as if it were a file — we can use » to extract tokens from it. Critical for parsing `/proc/[PID]/stat`.
`<vector>`	`std::vector` — a dynamic (growable) array. Holds the list of all processes.
`<filesystem>`	C++17 directory iteration. Used to scan `/proc` and find all PID directories.
`<iomanip>`	Formatted output (`setw`, `setprecision`). Controls column alignment and decimal places.
`<utmp.h>`	Linux-specific API for reading logged-in user sessions from the `utmp` database.

Header	Purpose
`<thread>`	`std::this_thread::sleep_for()` — pauses execution between screen refreshes.
`<chrono>`	Time durations (`std::chrono::milliseconds`). Works with `<thread>` for timing.
`<ncurses.h>`	The **ncurses** library for full-screen terminal UI. Replaces `cout` entirely with screen-painting functions.
`<dirent.h>`	POSIX directory reading (legacy). Included for compatibility.
`<pwd.h>`	Password database access. `getpwuid()` converts a numeric user ID (like 1000) to a username (like "john").
`<sys/stat.h>`	File metadata retrieval. `stat()` tells us who owns a `/proc/[PID]` directory (= who owns the process).
`<sys/types.h>`	System type definitions (`uid_t`, `pid_t`, etc.). Required by POSIX functions.
`<unistd.h>`	POSIX utilities. `sysconf()` gets the system memory page size.
`<algorithm>`	`std::sort()` — sorts the process list by CPU usage.
`<map>`	`std::map` — a key-value dictionary. Stores previous CPU tick counts per PID for delta calculations.

2.2 The Namespace Alias

```
namespace fs = std::filesystem;
```

Listing 2: Namespace alias for filesystem

3 Code Part 2: Data Structures (Structs)

Before writing any functions, we define **structs** — containers that group related pieces of data together. Think of them as labeled boxes where each compartment has a name and a type.

3.1 Structure 1: SystemSummary

```
struct SystemSummary {
    std::string upTime;     // e.g., "uptime: 5 days 10 hours 50 minutes"
    std::string loadAvg;    // e.g., "0.00 0.00 0.00"
};
```

Listing 3: SystemSummary struct

Explanation

This struct holds two pieces of text:

- `upTime` — A formatted string showing how long the system has been running
- `loadAvg` — A string with three numbers representing CPU load averages (1-minute, 5-minute, 15-minute)

Functions will **fill in** these fields, and the main loop will **display** them on screen. Using a struct keeps related data organized.

3.2 Structure 2: TaskStatus

```
struct TaskStatus {
    int total = 0;       // Total number of processes
    int running = 0;     // Processes currently executing on CPU (state "R")
    int sleeping = 0;    // Processes waiting for I/O or events (state "S" or "D")
    int stopped = 0;     // Processes paused by a signal (state "T")
    int zombie = 0;      // Dead processes whose parent hasn't collected exit (state "Z")
};
```

Listing 4: TaskStatus struct

Understanding Process States

Every process on Linux exists in one of several **states**:

State	Code	Meaning
Running	R	The process is actively using the CPU or waiting in the run queue
Sleeping	S	The process is waiting for an event (keyboard input, network data, timer) — *interruptible*
Disk Sleep	D	The process is waiting for disk I/O — *uninterruptible* (cannot be killed)
Stopped	T	The process has been paused by a signal (e.g., Ctrl+Z) or a debugger
Zombie	Z	The process has finished execution, but its parent process has not yet acknowledged its death by calling `wait()`. The zombie uses no CPU or memory, but its entry remains in the process table.

The `= 0` after each field sets a **default initial value**. Without it, the values would contain random garbage from memory.

3.3 Structure 3: MemoryStats

```
struct MemoryStats {
    float memTotal, memFree, memUsed, memBuffCache;
    float swapTotal, swapFree, swapUsed, availMem;
};
```

Listing 5: MemoryStats struct

Field-by-Field Explanation

Physical Memory (RAM) Fields:

- `memTotal` — Total physical RAM installed in the system (in MiB)
- `memFree` — RAM that is completely untouched and unused
- `memUsed` — RAM actively in use by programs (= Total − Available)
- `memBuffCache` — RAM used for disk caching (buffers + cached pages + reclaimable slab). This memory CAN be freed instantly if programs need it

Swap Memory Fields:

- `swapTotal` — Total swap space configured on disk
- `swapFree` — Unused swap space
- `swapUsed` — Swap space currently holding data moved from RAM
- `availMem` — The kernel's estimate of RAM available for new programs without swapping

3.4 Structure 4: CPUData (New)

```
struct CPUData {
    unsigned long long user, nice, system, idle, iowait, irq,
        softirq, steal;

    unsigned long long getTotalTime() const {
        return user + nice + system + idle + iowait + irq + softirq
            + steal;
    }

    unsigned long long getIdleTime() const {
        return idle + iowait;
    }
};
```

Listing 6: CPUData struct for storing CPU counters

3.5 Structure 5: Process

```
struct Process {
    int pid;
    std::string user;
    int priority;
    int nice;
    long virt;
    long res;
    double cpu_usage = 0.0;
    double mem_usage = 0.0;
    std::string command;
};
```

Listing 7: Process struct

4 Code Part 3: Step 1 — System Summary Functions

These functions read basic system information from `/proc` and the `utmp` database.

4.1 Function: getupTime() — Reading System Uptime

```
void getupTime(SystemSummary &stats) {
    std::ifstream uptimeFile("/proc/uptime");
    double total_seconds;
    std::string ans = "uptime: ";
    if (uptimeFile >> total_seconds) {
        int up_secs = (int)total_seconds;
        int days = up_secs / 86400;
        int hours = (up_secs % 86400) / 3600;
        int minutes = (up_secs % 3600) / 60;
        if (days > 0)  {ans += std::to_string(days); ans += " days
            ";}
        ans += std::to_string(hours); ans += " hours ";
        ans += std::to_string(minutes); ans += " minutes ";
        stats.upTime = ans;
    }
}
```

Listing 8: getupTime function

4.2 Function: getLoadAverage() — Reading CPU Load Averages

```
void getLoadAverage(SystemSummary &stats) {
    std::ifstream loadFile("/proc/loadavg");
    std::string one, five, fifteen;
    loadFile >> one >> five >> fifteen;
    stats.loadAvg = one + " " + five + " " + fifteen;
}
```

Listing 9: getLoadAverage function

What is Load Average?

Load average represents the average number of processes that are either:

- Currently running on the CPU, OR
- Waiting in the queue to run

Load Avg	Meaning (Single-Core CPU)
0.00	CPU is completely idle
0.50	CPU is 50% busy on average
1.00	CPU is exactly 100% utilized
2.00	CPU is overloaded — 1 process running + 1 waiting

For a **multi-core** CPU, multiply the threshold by the number of cores. A 4-core CPU with load average 4.00 is fully utilized.

4.3 Function: getActiveUserCount() — Counting Logged-In Users

```
int getActiveUserCount() {
    int count = 0;
    struct utmp *up;
    setutent();
    while ((up = getutent())) {
        if (up->ut_type == USER_PROCESS) {
            count++;
        }
    }
    endutent();
    return count;
}
```

Listing 10: getActiveUserCount function

4.4 Function: getDetailedTaskCount() — Counting Process States

```
void getDetailedTaskCount(TaskStatus &stats) {
    for (const auto& entry : fs::directory_iterator("/proc")) {
        std::string filename = entry.path().filename().string();

        if (entry.is_directory() && std::isdigit(filename[0])) {
            stats.total++;

            std::ifstream statFile(entry.path().string() + "/stat");
            if (statFile.is_open()) {
                std::string pid, comm, state;
                statFile >> pid >> comm >> state;

                if (state == "R") stats.running++;
                else if (state == "S" || state == "D")
                    stats.sleeping++;
                else if (state == "T") stats.stopped++;
                else if (state == "Z") stats.zombie++;
```

```
            }
        }
    }
}
```

Listing 11: getDetailedTaskCount function

4.5 Function: getMemoryAndSwap() — Reading Memory Statistics

```
void getMemoryAndSwap(MemoryStats &m) {
    std::ifstream file("/proc/meminfo");
    std::string key;
    float value;
    std::string unit;

    while (file >> key >> value >> unit) {
        if (key == "MemTotal:") m.memTotal = value / 1024.0;
        else if (key == "MemFree:") m.memFree = value / 1024.0;
        else if (key == "Buffers:") m.memBuffCache += value / 1024.0;
        else if (key == "Cached:") m.memBuffCache += value / 1024.0;
        else if (key == "SReclaimable:") m.memBuffCache += value /
            1024.0;
        else if (key == "SwapTotal:") m.swapTotal = value / 1024.0;
        else if (key == "SwapFree:") m.swapFree = value / 1024.0;
        else if (key == "MemAvailable:") m.availMem = value / 1024.0;
    }
    m.memUsed = m.memTotal - m.availMem;
    m.swapUsed = m.swapTotal - m.swapFree;
}
```

Listing 12: getMemoryAndSwap function

5 Code Part 4: Step 2 — Live CPU Usage Measurement

5.1 Why CPU Usage Requires Two Readings

The Delta Method — Most Important Concept

You **cannot** measure CPU usage at a single instant in time. Here is why:
Imagine asking "How fast is this car going?" You need **two position measurements** at two different times:

$$\text{Speed} = \frac{\text{Position}_2 - \text{Position}_1}{\text{Time}_2 - \text{Time}_1}$$

CPU usage works the same way. The kernel maintains **cumulative counters** (like an odometer) that tick upward since boot. To calculate the usage percentage, we must:

1. Take **Snapshot 1** of the counters
2. Wait a short time (e.g., 1 second)
3. Take **Snapshot 2** of the counters
4. Calculate: $\text{CPU\%} = 100 \times \frac{(\text{Total}_2 - \text{Total}_1) - (\text{Idle}_2 - \text{Idle}_1)}{\text{Total}_2 - \text{Total}_1}$

5.2 The CPUData Structure

```
struct CPUData {
    unsigned long long user, nice, system, idle, iowait, irq,
        softirq, steal;

    unsigned long long getTotalTime() const {
        return user + nice + system + idle + iowait + irq + softirq
            + steal;
    }

    unsigned long long getIdleTime() const {
        return idle + iowait;
    }
};
```

Listing 13: CPUData struct

5.2.1 Field-by-Field Explanation

Field	Meaning
`user`	Cumulative ticks spent running normal user-space programs (browsers, editors, scripts)
`nice`	Ticks spent running user programs with **lowered priority** ("niced" processes)
`system`	Ticks spent running kernel code (file I/O, memory management, networking)
`idle`	Ticks where the CPU had **nothing to do**
`iowait`	Ticks where the CPU was idle because it was **waiting for disk I/O**
`irq`	Ticks handling **hardware interrupts** (keyboard, network card, disk controller)
`softirq`	Ticks handling **software interrupts** (network packet processing, timers)
`steal`	Ticks "stolen" by a hypervisor (relevant in virtual machines / cloud)

5.2.2 The Two Helper Methods

- `getTotalTime()` — Returns the sum of ALL eight fields = total elapsed CPU time since boot
- `getIdleTime()` — Returns `idle + iowait`. When the CPU is waiting for disk, it is not doing useful work, so we count `iowait` as "idle"

5.3 Function: readCPUStats() — Reading /proc/stat

```
CPUData readCPUStats() {
    std::ifstream file("/proc/stat");
    std::string cpuLabel;
    CPUData data;
    file >> cpuLabel >> data.user >> data.nice >> data.system >>
        data.idle
          >> data.iowait >> data.irq >> data.softirq >> data.steal;
    return data;
}
```

Listing 14: readCPUStats function

5.4 Numerical Example of the Delta Calculation

```
SNAPSHOT 1 (time T):
  user=10000  system=5000  idle=80000  total=100000

         ... wait 1 second ...

SNAPSHOT 2 (time T+1):
```

```
  user=10200  system=5100  idle=80300  total=100800

DELTAS (differences):
  totalDelta = 100800 - 100000 = 800 ticks elapsed
  idleDelta  = 80300 - 80000  = 300 idle ticks

PERCENTAGES:
  Overall CPU  = 100 * (800 - 300) / 800    = 62.5%
  User %       = 100 * (10200 - 10000) / 800 = 25.0%
  System %     = 100 * (5100 - 5000) / 800   = 12.5%
  Idle %       = 100 * (80300 - 80000) / 800  = 37.5%
```

6 Code Part 5: Step 3 — Per-Process Information

6.1 The Process Structure

```cpp
struct Process {
    int pid;
    std::string user;
    int priority;
    int nice;
    long virt;
    long res;
    double cpu_usage = 0.0;
    double mem_usage = 0.0;
    std::string command;
};
```

Listing 15: Process struct

Field	Meaning
`pid`	PID: Unique process identifier. Every process gets a unique number.
`user`	USER: The username of the person/system account that started this process (e.g., "root", "john")
`priority`	PR: The kernel's internal scheduling priority. Lower number = higher priority.
`nice`	NI: User-adjustable priority hint. Range: -20 (highest priority) to $+19$ (lowest priority). Only root can set negative values.
`virt`	VIRT: Virtual memory in KiB. The total address space the process has *mapped* — includes shared libraries, memory-mapped files, and allocated-but-unused memory. Can be very large.

`res`	RES: Resident memory in KiB. The amount of physical RAM this process is *actually using right now*. This is the "real" memory consumption.
`cpu_usage`	%CPU: Percentage of CPU time used by this process (calculated via delta method)
`mem_usage`	%MEM: Percentage of total system RAM used by this process
`command`	COMMAND: The name of the executable (e.g., "firefox", "bash", "systemd")

6.2 Function: getProcessTicks() — Measuring Individual Process CPU Time

```
long long getProcessTicks(int pid) {
    std::ifstream statFile("/proc/" + std::to_string(pid) + "/stat");
    if (!statFile.is_open()) return 0;
    std::string line;
    std::getline(statFile, line);
    size_t lastParen = line.find_last_of(')');
    if (lastParen == std::string::npos) return 0;
    std::stringstream ss(line.substr(lastParen + 2));
    std::string dummy;
    long long utime, stime;
    for(int i=0; i<11; i++) ss >> dummy;
    ss >> utime >> stime;
    return utime + stime;
}
```

Listing 16: getProcessTicks function

6.3 Function: getUsername() — Converting User IDs to Names

```
std::string getUsername(uid_t uid) {
    struct passwd *pw = getpwuid(uid);
    return pw ? pw→pw_name : std::to_string(uid);
}
```

Listing 17: getUsername function

6.4 Function: getProcessList() — The Main Process Data Collector

```
std::vector<Process> getProcessList(float totalMemMiB) {
    std::vector<Process> procs;
    for (const auto& entry : fs::directory_iterator("/proc")) {
        std::string pidStr = entry.path().filename().string();
        if (std::isdigit(pidStr[0])) {
            Process p;
            p.pid = std::stoi(pidStr);


```

```
            // 1. Get User
            struct stat info;
            if (::stat(entry.path().c_str(), &info) == 0) {
                p.user = getUsername(info.st_uid);
            }

            // 2. Get PR and NI from /proc/[PID]/stat
            std::ifstream statFile("/proc/" + pidStr + "/stat");
            std::string line;
            std::getline(statFile, line);
            size_t lastParen = line.find_last_of(')');
            std::stringstream ss(line.substr(lastParen + 2));

            std::string dummy;
            for(int i=0; i<15; i++) ss >> dummy;
            ss >> p.priority >> p.nice;

            // 3. Get Memory from /proc/[PID]/statm
            std::ifstream statmFile("/proc/" + pidStr + "/statm");
            long size, resident;
            statmFile >> size >> resident;
            long pageSizeKiB = sysconf(_SC_PAGESIZE) / 1024;
            p.virt = size * pageSizeKiB;
            p.res = resident * pageSizeKiB;
            p.mem_usage = (p.res / 1024.0 / totalMemMiB) * 100.0;

            // 4. Command
            std::ifstream cmdFile("/proc/" + pidStr + "/comm");
            std::getline(cmdFile, p.command);

            procs.push_back(p);
        }
    }
    return procs;
}
```

Listing 18: getProcessList function

7 Code Part 6: The main() Function — Where Everything Comes Together

7.1 Part 1: Initializing ncurses (Terminal UI Setup)

```
int main() {

    // Initialize ncurses
    initscr();              // Start ncurses mode
    noecho();               // Don't echo keypresses to the screen
    curs_set(0);            // Hide the blinking cursor
    nodelay(stdscr, TRUE); // Make getch() "non-blocking"
    keypad(stdscr, TRUE);  // Enable arrow keys/special keys
```

Listing 19: ncurses initialization

7.2 Part 2: One-Time Setup — Getting Total System Memory

```
    // Get Total System Memory (needed for %MEM calculation)
    float totalSystemMem = 0;
    std::ifstream memInfo("/proc/meminfo");
    std::string key;
    float value;
    while (memInfo >> key >> value) {
        if (key == "MemTotal:") {
            totalSystemMem = value / 1024.0; // Convert KB to MiB
            break;
        }
    }
```

Listing 20: Reading total system memory once

7.3 Part 3: Declaring Main Loop Variables

```
    bool running = true;
    int refreshMillis = 1000;
    CPUData s1 = readCPUStats(); // Initial snapshot
    std::map<int, long long> prevProcTicks;
    CPUData prevSystem = readCPUStats();
    int maxdisplay = 15;
    bool sortByMemory = false;
```

Listing 21: Main loop variable declarations

7.3.1 Variable-by-Variable Explanation

Variable	Purpose

`running`	Boolean flag controlling the main loop. When the user presses 'q', this is set to `false`, which exits the loop.
`refreshMillis`	Time in milliseconds between screen refreshes. Default = 1000 (1 second). The user can change this by pressing 'd'.
`s1`	The "previous" CPU snapshot. Used to calculate overall CPU% by comparing with a new snapshot each cycle.
`prevProcTicks`	A **dictionary** (key-value store) mapping each PID to its CPU tick count from the **previous** refresh cycle. This allows us to compute per-process CPU% using the delta method.
`prevSystem`	The "previous" system-wide CPU snapshot, used as the denominator for per-process CPU% calculations.

7.4 Part 4: The Main Loop — Section A: User Input Handling

```
    while (running) {
        // A. Check for user input
        int ch = getch();
        if (ch == 'q' || ch == 'Q') {
            running = false;
        }
```

Listing 22: User input handling

7.4.1 Explanation

- `while (running)` — Loop continues as long as `running` is `true`
- If the user pressed 'q' or 'Q', we set `running = false`. The loop finishes its current iteration and then stops.

7.4.2 The 'd' Key: Change Refresh Delay

```
        else if (ch == 'd' || ch == 'D') {
        // 1. Temporary disable non-blocking to get user input
        nodelay(stdscr, FALSE);
        echo();
        curs_set(1);

        mvprintw(0, 0, "Change delay from %.1fs to: ", refreshMillis
            / 1000.0);
        clrtoeol();

        char input[10];
        getnstr(input, 9);

        try {
            float newDelay = std::stof(input);
```

```
            if (newDelay > 0) {
                refreshMillis = static_cast<int>(newDelay * 1000);
            }
        } catch (...) {
            // If user enters non-numeric, just keep the old delay
        }

        // 2. Restore TUI settings
        noecho();
        nodelay(stdscr, TRUE);
        curs_set(0);
    }
```

Listing 23: Change Delay Feature

7.4.3 The 'k' Key: Kill Process by PID

```
        else if (ch == 'k' || ch == 'K'){
            // 1. Switch to input mode
            nodelay(stdscr, FALSE);
            echo();
            curs_set(1);

            // 2. Ask for PID
            mvprintw(0, 0, "Enter PID to kill: ");
            clrtoeol();
            refresh();

            char input[10];
            getnstr(input, 9);

            int pid = 0;
            bool success = false;

            try {
                pid = std::stoi(input);

                // 3. Try killing process
                if (kill(pid, SIGTERM) == 0) {
                    success = true;
                }
            } catch (...) {
                // invalid input handled below
            }

            // 4. Show result message (safe row)
            mvprintw(2, 0, "                                                    ");
            if (success) {
                mvprintw(maxdisplay+19, 0, "Process %d killed successfully!", pid);
            } else {
                mvprintw(maxdisplay+19, 0, "Failed to kill process or invalid PID!");
            }

            refresh();
```

```

            // 5. Pause so user can SEE the message
            std::this_thread::sleep_for(std::chrono::milliseconds(1500));

            // 6. Restore ncurses mode
            noecho();
            nodelay(stdscr, TRUE);
            curs_set(0);
        }
```

Listing 24: Kill Process Feature

7.4.4 The 'n' Key: Change Number of Displayed Processes

```
        else if (ch == 'n' || ch == 'N')
        {
            // 1. Enable input mode
            nodelay(stdscr, FALSE);
            echo();
            curs_set(1);

            // 2. Ask user
            mvprintw(0, 0, "Change number of processes to: ");
            clrtoeol();
            refresh();

            char input[10];
            getnstr(input, 9);

            try {
                int newLimit = std::stoi(input);

                if (newLimit > 0) {
                    maxdisplay = newLimit;

                    mvprintw(maxdisplay+18, 0, "Now showing %d
                        processes.", maxdisplay);
                } else {
                    mvprintw(maxdisplay+18, 0, "Invalid number!");
                }
            } catch (...) {
                mvprintw(maxdisplay+18, 0, "Invalid input!");
            }

            refresh();
            std::this_thread::sleep_for(std::chrono::milliseconds(1200));

            // 3. Restore mode
            noecho();
            nodelay(stdscr, TRUE);
            curs_set(0);
        }
```

Listing 25: Change Number of Displayed Processes Feature

7.4.5 The 'm' Key: Toggle Sort by Memory Usage

```
        else if (ch == 'm' || ch == 'M')
        {
            sortByMemory = !sortByMemory;  // toggle

            // Show current mode
            mvprintw(2, 0, "                                                        ");  // clear line
            if (sortByMemory) {
                mvprintw(maxdisplay+18, 0, "Sorting by MEMORY usage.");
            } else {
                mvprintw(maxdisplay+18, 0, "Sorting by CPU usage.");
            }

            refresh();
            std::this_thread::sleep_for(std::chrono::milliseconds(1200));
        }
```

Listing 26: Toggle Sort by Memory Usage Feature

7.5 Part 5: The Main Loop — Section B: Data Collection

7.5.1 Step 1 Data: System Summary

```
        // B. Update Data

        //Get Step-1 Data
        SystemSummary stats;
        getupTime(stats);
        int userCount = getActiveUserCount();
        getLoadAverage(stats);
        TaskStatus taskStats;
        getDetailedTaskCount(taskStats);
        MemoryStats m = {0};
        getMemoryAndSwap(m);
```

Listing 27: Collecting Step 1 data

- Creates fresh struct instances each iteration (we want current data, not stale data)
- Calls all five Step 1 functions to populate the structs
- `MemoryStats m = {0}` — The `{0}` initializes **all fields to zero**.

7.5.2 Step 2 Data: Overall CPU Percentage

```
        //Get Step-2 Data
        CPUData s2 = readCPUStats();
        unsigned long long totalDelta = s2.getTotalTime() - s1.getTotalTime();
        unsigned long long idleDelta = s2.getIdleTime() - s1.getIdleTime();
```

```
        double usage = 100.0 * (totalDelta - idleDelta) / totalDelta;
        double userPerc = 100.0 * (s2.user - s1.user) / totalDelta;
        double sysPerc = 100.0 * (s2.system - s1.system) /
            totalDelta;
        double idlePerc = 100.0 * (s2.idle - s1.idle) / totalDelta;
```

Listing 28: Collecting Step 2 data — CPU percentages

7.5.3 Explanation with Formulas

1. `s2 = readCPUStats()` — Takes a new CPU counter snapshot
2. **Total Delta** (total ticks elapsed since last refresh):

$$\texttt{totalDelta} = \texttt{s2.getTotalTime()} - \texttt{s1.getTotalTime()}$$

3. **Idle Delta** (idle ticks during this interval):

$$\texttt{idleDelta} = \texttt{s2.getIdleTime()} - \texttt{s1.getIdleTime()}$$

4. **Overall CPU Usage:**

$$\texttt{usage} = 100 \times \frac{\texttt{totalDelta} - \texttt{idleDelta}}{\texttt{totalDelta}}$$

5. **User Mode Percentage:**

$$\texttt{userPerc} = 100 \times \frac{\texttt{s2.user} - \texttt{s1.user}}{\texttt{totalDelta}}$$

6. **System Mode Percentage:**

$$\texttt{sysPerc} = 100 \times \frac{\texttt{s2.system} - \texttt{s1.system}}{\texttt{totalDelta}}$$

7. **Idle Percentage:**

$$\texttt{idlePerc} = 100 \times \frac{\texttt{s2.idle} - \texttt{s1.idle}}{\texttt{totalDelta}}$$

7.5.4 Step 3 Data: Per-Process Information

```
        //Get Step-3 Data
        CPUData currSystem = readCPUStats();
        unsigned long long systemDelta =
            currSystem.getTotalTime() - prevSystem.getTotalTime();
        std::vector<Process> currentProcs =
            getProcessList(totalSystemMem);
        for (auto& p : currentProcs) {
            long long currTicks = getProcessTicks(p.pid);
            if (prevProcTicks.count(p.pid) && systemDelta > 0) {
                p.cpu_usage =
```

```
                    (100.0 * (currTicks - prevProcTicks[p.pid])) / systemDelta;
            }
            prevProcTicks[p.pid] = currTicks;
        }
        std::sort(currentProcs.begin(), currentProcs.end(),
            [](const Process& a, const Process& b) {
                return a.cpu_usage > b.cpu_usage;
            });
```

Listing 29: Collecting Step 3 data — per-process information

7.6 Part 6: The Main Loop — Section C: Screen Rendering

```
        // C. Render to Screen
        clear(); // Clear the previous frame

        // Step-1 output printing
        mvprintw(0, 0, "%s\n", stats.upTime.c_str());
        mvprintw(1, 0, "Number of users: %d\n", userCount);
        mvprintw(2, 0, "load average: %s\n", stats.loadAvg.c_str());
        mvprintw(3, 0,
            "Tasks: %d total, %3d running, %3d sleeping, %3d stopped, %3d zombie\n",
            taskStats.total, taskStats.running, taskStats.sleeping,
            taskStats.stopped, taskStats.zombie);
        mvprintw(4, 0,
            "MiB Mem : %8.1f total, %8.1f free, %8.1f used, %8.1f buff/cache\n",
            m.memTotal, m.memFree, m.memUsed, m.memBuffCache);
        mvprintw(5, 0,
            "MiB Swap: %8.1f total, %8.1f free, %8.1f used. %8.1f avail Mem\n",
            m.swapTotal, m.swapFree, m.swapUsed, m.availMem);

        // Step-2 Output printing
        mvprintw(6, 0,
            "%%Cpu(s): %5.1f us, %5.1f sy, %5.1f ni, %5.1f id\n",
            userPerc, sysPerc,
            100.0 * (s2.nice - s1.nice) / totalDelta, idlePerc);

        // Step-3 Output printing
        mvprintw(8, 0, "List of Processes and their Info");
        mvprintw(10, 0,
            "%-7s %-10s %-4s %-4s %-8s %-8s %-6s %-6s %-15s",
            "PID", "USER", "PR", "NI", "VIRT", "RES",
            "%CPU", "%MEM", "COMMAND");

        for (int i = 0; i < 15 && i < currentProcs.size(); ++i) {
            const auto& p = currentProcs[i];
            mvprintw(11 + i, 0,
                "%-7d %-10s %-4d %-4d %-8ld %-8ld %-6.1f %-6.1f %-15s",
                p.pid,
                p.user.substr(0, 9).c_str(),
                p.priority,
                p.nice,
```

```
                    p.virt,
                    p.res,
                    p.cpu_usage,
                    p.mem_usage,
                    p.command.substr(0, 14).c_str());
            }
            mvprintw(maxdisplay+20, 0, "Press 'q' to exit.");

            refresh(); // Push all changes to the actual terminal
```

Listing 30: Screen rendering with ncurses

7.7 Part 7: The Main Loop — Section D: Preparing for Next Cycle

```
            // D. Prepare for next iteration
            s1 = s2;
            prevSystem = currSystem;
            std::this_thread::sleep_for(
                std::chrono::milliseconds(refreshMillis));
        }
```

Listing 31: Preparing for next iteration

7.8 Part 8: Program Cleanup

```
        // Cleanup ncurses before exiting
        endwin();
        return 0;
}
```

Listing 32: Program cleanup and exit

8 Compilation and Execution

8.1 Build Command

```
g++ minitopImplementation.cpp -lncurses -o minitop
```

Listing 33: Compilation command

Flag	Meaning
`g++`	The GNU C++ compiler
`-std=c++17`	Use C++17 standard (required for `<filesystem>`)
`-o minitop`	Name the output executable `mytop`
`minitopImplementation.cpp`	The source file to compile
`-lncurses`	Link against the ncurses library
`-lstdc++fs`	Link against the filesystem library (older compilers only)

8.2 Run Command

```
./minitop
```

Listing 34: Running the program

8.3 Runtime Controls

Key	Action
`q` or `Q`	Quit the program and restore the terminal
`d` or `D`	Change the refresh delay (prompts for new value in seconds)
`k` or `K`	Kill a process by entering its PID
`n` or `N`	Change the number of processes displayed
`m` or `M`	Toggle sorting between CPU usage and memory usage

Thank You